MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Exploits

Unit 8 · Ethical Hacking (10-151-160)

MID-STATE TECHNICAL COLLEGE

Learning Outcomes

By the end of this unit, you will be able to:



Distinguish exploits from vulnerabilities

Define exploit, vulnerability, and payload, and explain how each relates to the others.



Research exploits from scan results

Turn vulnerabilities and service versions into matching exploits using public databases.



Use exploit frameworks

Describe how Metasploit organizes modules, payloads, and encoders into a repeatable workflow.



Gain access against an authorized target

In the lab, use an exploit to obtain a foothold and understand what a payload gives an attacker.



Detect and mitigate exploits

Configure antivirus, recognize evasion, and apply the controls that break the attack chain.

Warm-Up: Recent Exploits in the News

Before we start, let us connect this unit to what is happening right now.

- Bring a **recent breach, exploit, or zero-day** you have seen in the news or a feed.
- For each one, ask: **where was the flaw**, and which stage of the attack did it enable?
- Map it to a framework (**MITRE ATT&CK** or the kill chain) so the control gap is visible.
- Why it matters: exploit knowledge decays fast — staying current is part of the job, not extra credit.

Feeds worth following: Exploit-DB, the CISA Known Exploited Vulnerabilities catalog, Krebs on Security, SANS Internet Storm Center.

LESSON 1

Exploits vs. Vulnerabilities



What Is an Exploit?

An exploit is a way in. It takes advantage of a weakness for the attacker's benefit.

- **Vulnerability** — a weakness that can be taken advantage of: unpatched software, a misconfiguration, weak credentials, or a person. The open door.
- **Exploit** — the way in that takes advantage of that weakness. The act of walking through the door.
- **Payload** — what runs once access is gained, such as a remote shell. The exploit gets you in; the payload is what it delivers.



Defender's lens

You remediate **vulnerabilities**, not exploits. Close the weakness and every exploit that targets it retires at once.

Exploit or Vulnerability?

A quick sorting drill — one is an action, the other is a condition.

Exploits (actions)

- Privilege escalation
- Reverse shell
- SQL injection
- Buffer overflow

Vulnerabilities (conditions)

- Outdated IIS version running
- Missing data encryption
- Missing authentication on a critical function
- Trusting unvalidated user input

The test: if it is something an attacker does, it is an exploit; if it is a flaw sitting in the system, it is a vulnerability.

LESSON 2

Finding Exploits



Classifying What You Find

Before you go looking, know the categories — they tell you how much reliability and risk to expect.

- **Known** — observed and documented in the wild: real attacks, open-source proof-of-concept code, and the **CVEs** that track them.
- **Unknown** — **zero-days**: no patch and no public signature yet. Rare and valuable.
- **Manual** — working a known weakness by hand against a specific target.
- **Automated** — running the steps through a tool, script, or framework.



SECURITY FOCUS

Most successful attacks are not clever zero-days — they reuse **known** exploits against systems that were never patched.

Why it matters: Timely patching removes the opportunity for the entire known-exploit category.

Where to Research Exploits

Take the vulnerabilities and versions from your scans and look them up here:

Source	What it gives you	Use it to
CVE / NVD	Catalog of known vulnerabilities with severity scores	Identify the flaw
Exploit-DB	Archive of public exploit and proof-of-concept code	Find the exploit
AttackerKB	Community view of which vulns are actually exploited	Judge the risk
OWASP Top Ten / CWE	Categories of weakness that exploits target	Know what to look for

CVEs are a leading indicator of new potential exploits — a new CVE often means exploit code is coming.

Use Recon to Pivot to Exploitation

Exploitation starts with the recon and scanning you already practiced — in this order:

- **1. Know open services and ports** — from `nmap`, `masscan`, or Legion.
- **2. Find versions** — a service banner or version is what you search against.
- **3. Find vulnerabilities** — Nessus and OWASP ZAP flag the weaknesses.
- **4. Find exploits** — Exploit-DB, AttackerKB, and the Metasploit `search` command.



Blue Team

Every one of these steps is noisy. Scan detection, version-probe alerts, and IDS signatures give defenders the chance to see an attack **before** exploitation ever begins.

LESSON 3

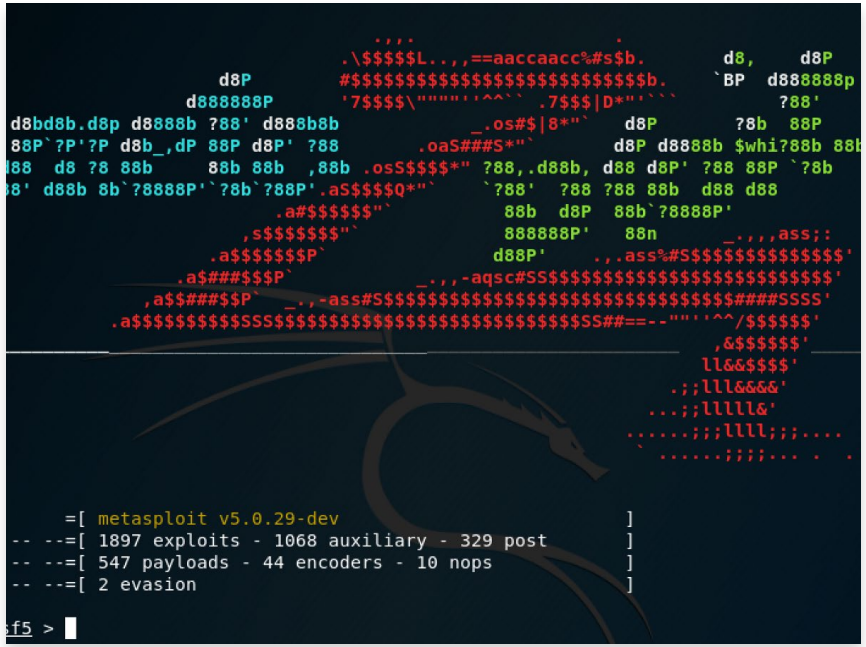
Frameworks & Payloads



Exploit Frameworks

A framework packages exploits, payloads, and encoders into one repeatable workflow.

- **Metasploit** — the best-known open-source framework. Its console (`msfconsole`) searches modules and manages sessions. Standard in training and pen testing.
- **Cobalt Strike** — a commercial red-team platform. Its source leaked in 2020, and cracked copies are now widely abused by criminals — a frequent sight in real breach reports.

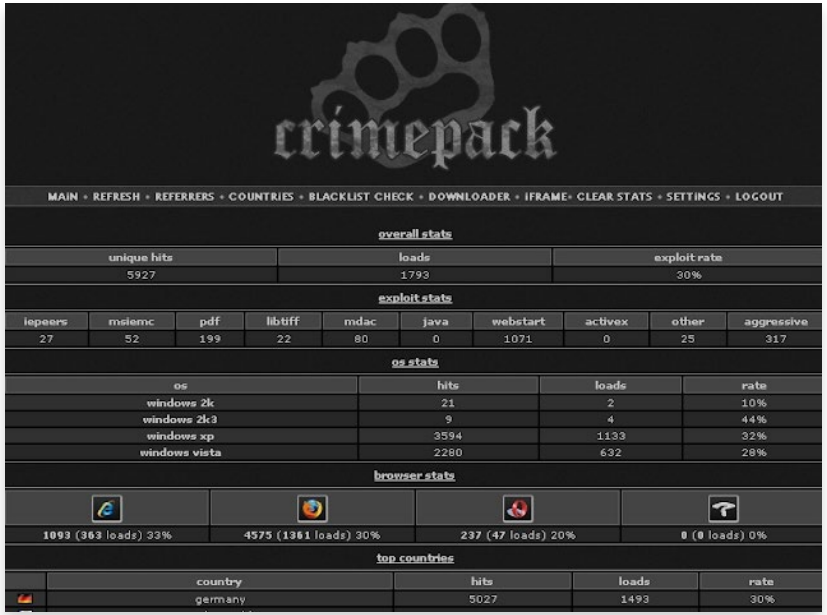


The console banner reports how many exploits, payloads, and encoders are loaded — a sense of the framework's scale.

Exploit & Malvertising Kits

At the criminal end, exploitation is packaged and sold as a service.

- **Rig**, **Neutrino**, and **Magnitude** — kits that compromise sites or buy malicious ads to deliver payloads to visitors automatically.
- **Malvertising** — ads that attack a user just by loading, or after a click.
- **CrimePack** — a paid kit with a dashboard tracking hits, loads, and a per-exploit success rate.



A reminder that attackers run this like a business — with metrics — which is exactly why staying current matters.

Metasploit vs. Meterpreter

Related, but not the same thing — and the difference explains how a modern payload behaves.

Metasploit — the framework

- A toolkit to find modules, build payloads, and manage sessions
- Searchable library of exploits and payloads (`msfconsole`)
- Think of it as the workshop

Meterpreter — a payload

- An interactive payload delivered by Metasploit that opens a session
- Example module: `payload/windows/meterpreter/reverse_tcp`
- Think of it as one tool from the workshop

Why Meterpreter is hard to catch: it uses a reverse (outbound) connection to slip past firewalls and runs in memory, writing little to disk — so file-only antivirus misses it.

Payload Creation — How It Works

Tools such as msfvenom build a standalone payload file. Three inputs decide what it does:

- **Payload type** — what runs on the target, e.g. a Windows Meterpreter reverse-TCP payload.
- **Listener address and port** — where the target calls back to; the default port `4444` is itself a well-known tell.
- **Output format** — how the payload is packaged, such as a Windows executable.



SECURITY FOCUS

A reverse connection exploits the fact that firewalls usually allow outbound traffic. **Egress filtering** and watching for odd outbound sessions is the counter.

Why it matters: Blocking and monitoring what leaves your network catches the callback the payload depends on.

LESSON 4

The Attack Chain



Remember the Kill Chain

Exploitation is one stage in a sequence that begins with the recon you already know.



Break any single link — delivery, exploitation, or command-and-control — and the attack fails.

Weaponization & Delivery

Weaponization

- Build a payload from what recon revealed
- Two parts: the `exploit` (a foothold) and the RAT or backdoor (the payload)
- Blue Team: threat intel on tooling, before anything lands

Delivery

- Choose how the payload reaches the target
- Email, malicious URL, attachment, USB, watering hole, DNS poisoning
- Blue Team: email and web filtering, removable-media policy, user awareness

Key idea: delivery is still pre-compromise — the strongest, cheapest defenses live here, before the exploit ever runs.

Exploitation

The exploit runs on the target — user-initiated, command-and-control-initiated, or hands-on.

- **Victim-initiated** — the user opens the attachment or visits the page.
- **C2-initiated** — the attacker triggers it from a command-and-control channel.
- **Hands-on** — run in person or over an existing remote session.
- **Most exploits only work on unpatched and outdated systems** — current systems simply are not vulnerable to them.



SECURITY FOCUS

Patching and hardening remove the opportunity; **EDR** catches the execution if one slips through.

Why it matters: An attacker needs an unpatched flaw to land — take it away and the exploit has nothing to stand on.

LESSON 5

Antivirus, Evasion & Defense



Ethical Hacking · Exploits

How Antivirus Decides What Is Malicious

To configure it well — and to see why some payloads slip past — know how it decides.

- **Signature / hash matching** — compare against a database of known malware. Fast and accurate for known samples, blind to new ones.
- **Heuristic analysis** — flag code that structurally resembles known malware families.
- **Behavioral detection (EDR)** — watch what a program does at runtime and alert on the behavior, regardless of signature.



Configure it to catch this unit's payloads

Enable real-time **memory scanning**, behavior monitoring, and blocking of unsigned executables — not file scanning alone.

Antivirus Evasion — Awareness

Why evasion exists: signature detection can be defeated by changing the payload's fingerprint. We study it to defend against it.

- **Obfuscation** changes the payload's signature while its behavior stays the same. **Counter:** behavioral/EDR detection ignores the fingerprint.
- **Encoding / packing** hides the payload until runtime. **Counter:** memory scanning inspects it after it unpacks.
- **In-memory / fileless** execution leaves no file to scan. **Counter:** memory and process-behavior monitoring.
- **Living off the land** abuses trusted built-in tools. **Counter:** baseline normal use and alert on anomalies.

Rules of engagement: these techniques are practiced only on isolated lab systems you are authorized to test — never on systems you do not own or have written permission to assess.

Measuring Detection: VirusTotal

VirusTotal shows how many antivirus engines detect a sample — and what they call it.

- Upload a sample and you get a **detection ratio** across dozens of engines.
- A plain payload is flagged by most engines; an **obfuscated** one by noticeably fewer.
- This is why “the antivirus will catch it” is not, by itself, a complete defense.

SECURITY FOCUS

A low detection rate on one engine is **not** a reason to trust a file — it is a reason to layer behavioral detection, egress control, and patching behind it.

Why it matters: Defense in depth means an attacker must beat every layer; you only need to win at one.

LIVE DEMO *Upload a lab-generated sample to VirusTotal and compare plain vs. obfuscated detection ratios.*

Detecting & Mitigating Exploits

An exploit needs a weakness, a way in, and a way to run undetected. Remove any one and it fails.

- **Patch and update** — retires whole classes of exploits at once; the highest-value control.
- **Harden configurations** — remove defaults, disable unused services, shrink the attack surface.
- **Filter egress** — reverse shells need outbound connections; restrict and watch them.
- **EDR and allow-listing** — catch execution that signature antivirus misses.
- **Monitor and correlate** — feed a SIEM and map events to MITRE ATT&CK.



SECURITY FOCUS

A critical flaw in a legacy device with no patch and no replacement cannot be closed. Isolate it on its own VLAN, wrap it in monitoring, and record it as an **accepted risk** with a named owner and review date.

Why it matters: Turn an exposure you cannot fix into a documented control requirement, not a closed ticket. Scan output: TLP:AMBER.

From Attacker to Defender: Why We Learn This

We learn the attacker's methods so an enterprise security team can anticipate and stop them.

In the enterprise

- Pen testing validates defenses before a real attacker finds the gap
- Red-team findings become detection rules and patch priorities
- Exploit awareness sharpens incident response and threat hunting

Discussion

- When are these techniques necessary — and when would you absolutely not use them?
- How would you detect the payloads we built?
- The line: **never without written authorization and defined scope.**

The ethic: the same skill is a professional service with permission and a crime without it. Scope and authorization are what separate the two.

Hands-On Lab: Windows Payload

Kali lab · authorized Windows VM only · graded on a working configuration and the defensive write-up



- Generate a Windows reverse-TCP Meterpreter payload with msfvenom and explain each option.
- Submit it to VirusTotal; record the detection ratio and what engines name it.
- Rebuild it through Veil to obfuscate, resubmit, and compare the new ratio.
- Set up a listener, run the payload on the authorized Windows VM, and confirm the session.
- Write up which detection and prevention controls would have stopped each stage.

The defensive write-up is graded alongside the technical steps — the point is to understand the adversary, then stop them.

Practice & Midterm Challenge

Build fundamentals in the guided rooms, then put them to work in the team challenge.

- **Vulnerabilities 101** (TryHackMe) — connect scan findings to exploit research.
- **AttackerKB** (TryHackMe) — judge which vulnerabilities are worth exploiting.
- **Midterm CTF** — self-chosen teams, recon → vulnerability → exploit on assigned boxes.
- Each member completes at least two challenges; highest team score wins.

Hack The Box and TryHackMe are equally valid — use whichever your team prefers. Deliverables: slides, presentation, score sheet.



Key Takeaways

- A vulnerability is the weakness; the exploit is the way in; the payload is what runs.
- Research exploits from scan results — CVE/NVD, Exploit-DB, and AttackerKB.
- Metasploit is the framework; Meterpreter is one payload it can deliver.
- Evasion defeats signatures — so defense relies on behavior, memory, and egress.
- Patching removes the opportunity; most exploits only work on unpatched systems.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 8 · Exploits

MID-STATE TECHNICAL COLLEGE